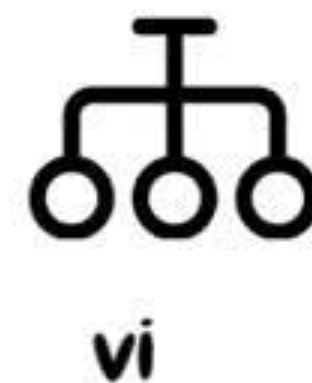
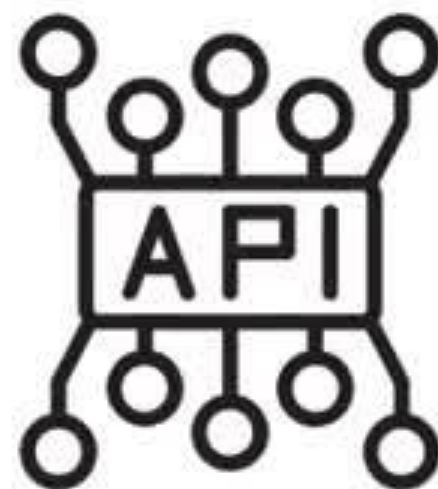
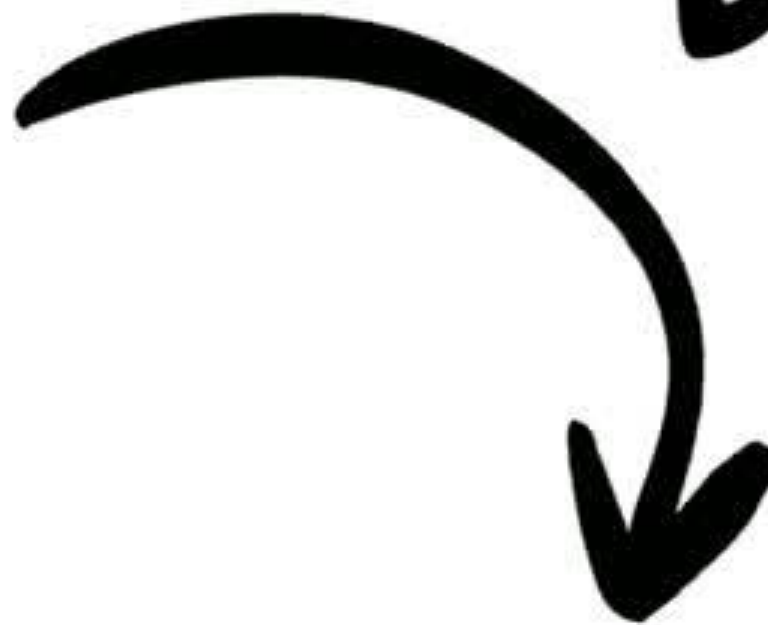


A Human-Like Agent For Screen-Aware Task Automation





Depend o



Title of the individual component: Visual Perception

Specialization: Software Engineering



Individual Title: Visual Perception

💡 Proven Gap / Creative Solution

- Current automation tools (Selenium, Appium, UIAutomator) require internal access (DOM, APIs, app-level hooks) → unsuitable for black-box testing.
- No reliable solution exists for external, human-like screen observation.
- **Creative solution:** external camera + computer vision + OCR + VLM → enables cross-platform, non-intrusive, black-box test automation.

🔍 Knowledge Gap / Problem Definition

- Tools lack ability to perceive screens visually like humans → fail in legacy or restricted environments.
- Existing OCR/UI detection models struggle with real-time noisy inputs (glare, blur, distortions).

⚠️ Shortcomings of Existing Systems & Proposed Solution

Shortcomings

- require internal access
- Limited to specific platforms
- OCR/UI detection in existing tools not optimized for real-time noisy inputs

Proposed Solution

- no need for system-level or app-level access.
- Cross-platform black-box automation
- Preprocessing + Computer Vision + OCR for robust recognition.

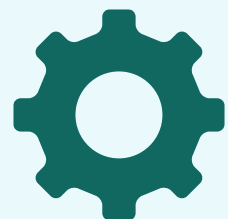
Individual Title: Visual Perception

Key Domains of Knowledge



- Computer Vision – OpenCV, deep learning models for object/element detection.
- OCR – Tesseract, EasyOCR, PaddleOCR for text extraction across variable fonts/colors.
- VLM (Vision-Language Models) – CogAgent, ShowUI, LLaVA for grounding perception with language reasoning.

Latest Technologies



- Real-time external camera feed capture.
- Preprocessing pipelines for clarity improvement.
- Region of Interest (ROI)-based change detection to minimize processing overhead.
- VLM integration for semantic understanding of screen states.

Individual Title: Visual Perception

Evaluation & Validation



- Accuracy of UI element detection $\geq 90\%$.
- OCR success rate across varied interfaces.
- Latency Testing → Evaluate frame-to-JSON conversion time (goal $< 500\text{ms}$).
- Robustness → Test under different screen resolutions, lighting conditions, and angles.

Data & Ethical Clearance

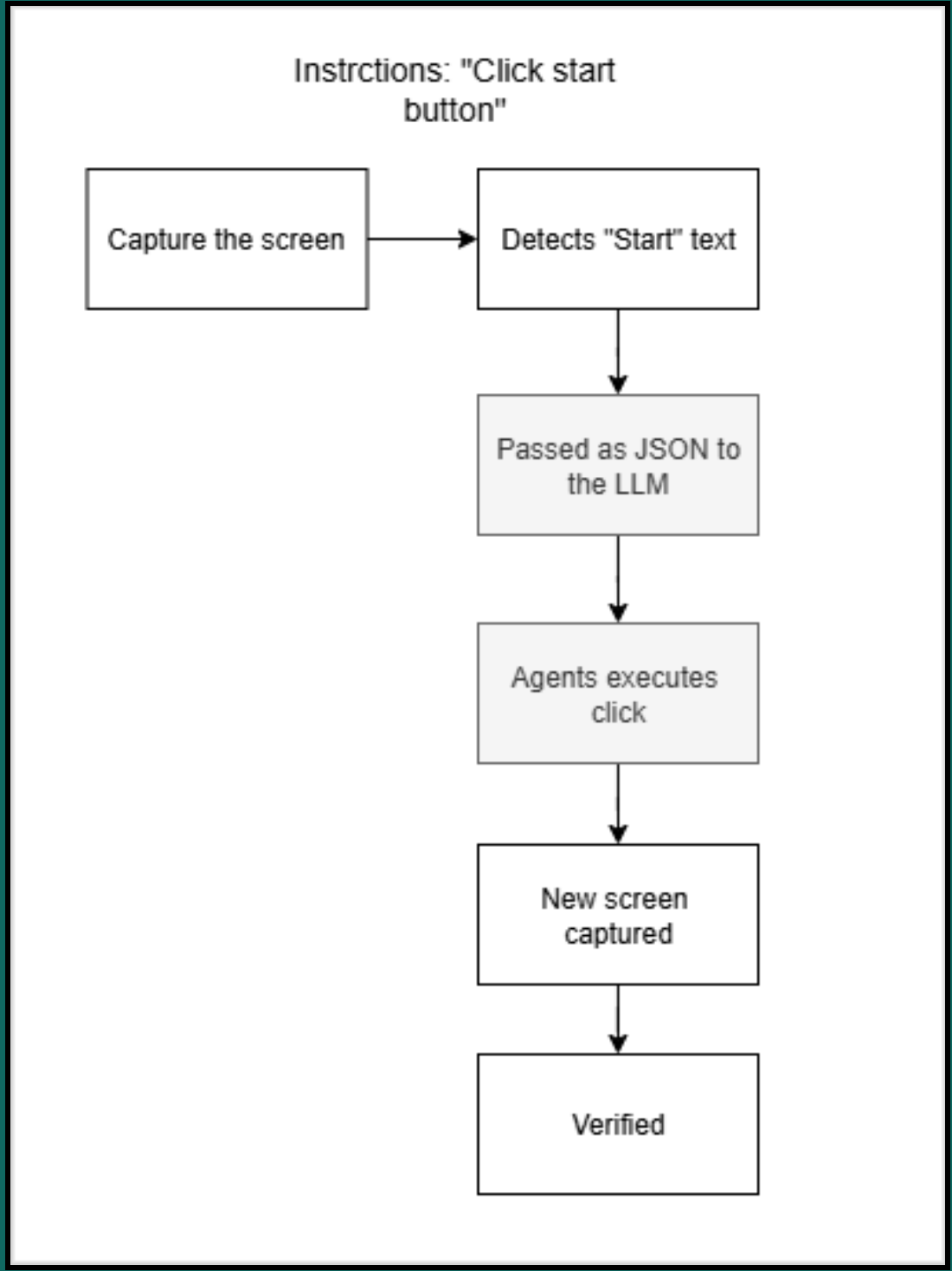
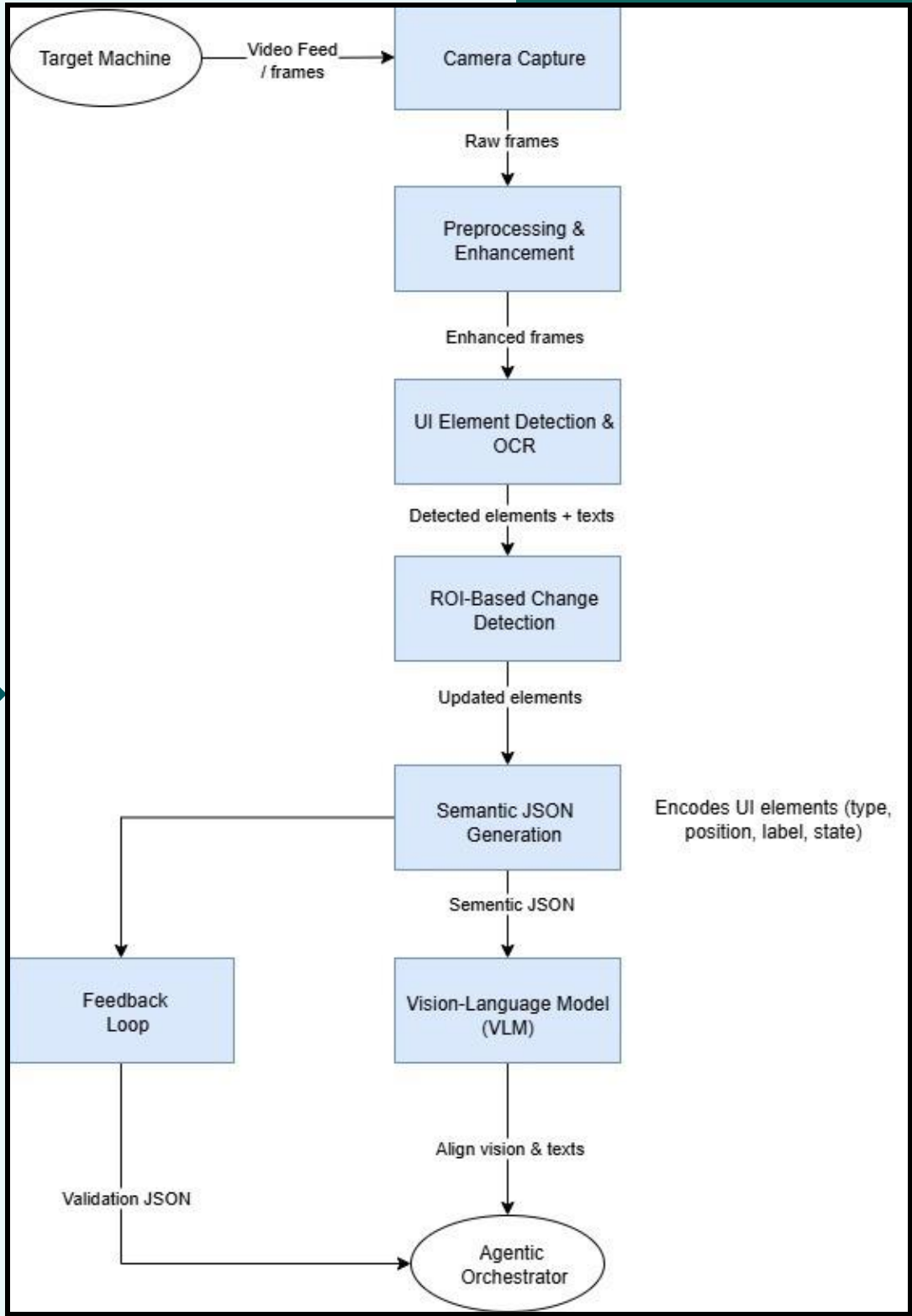
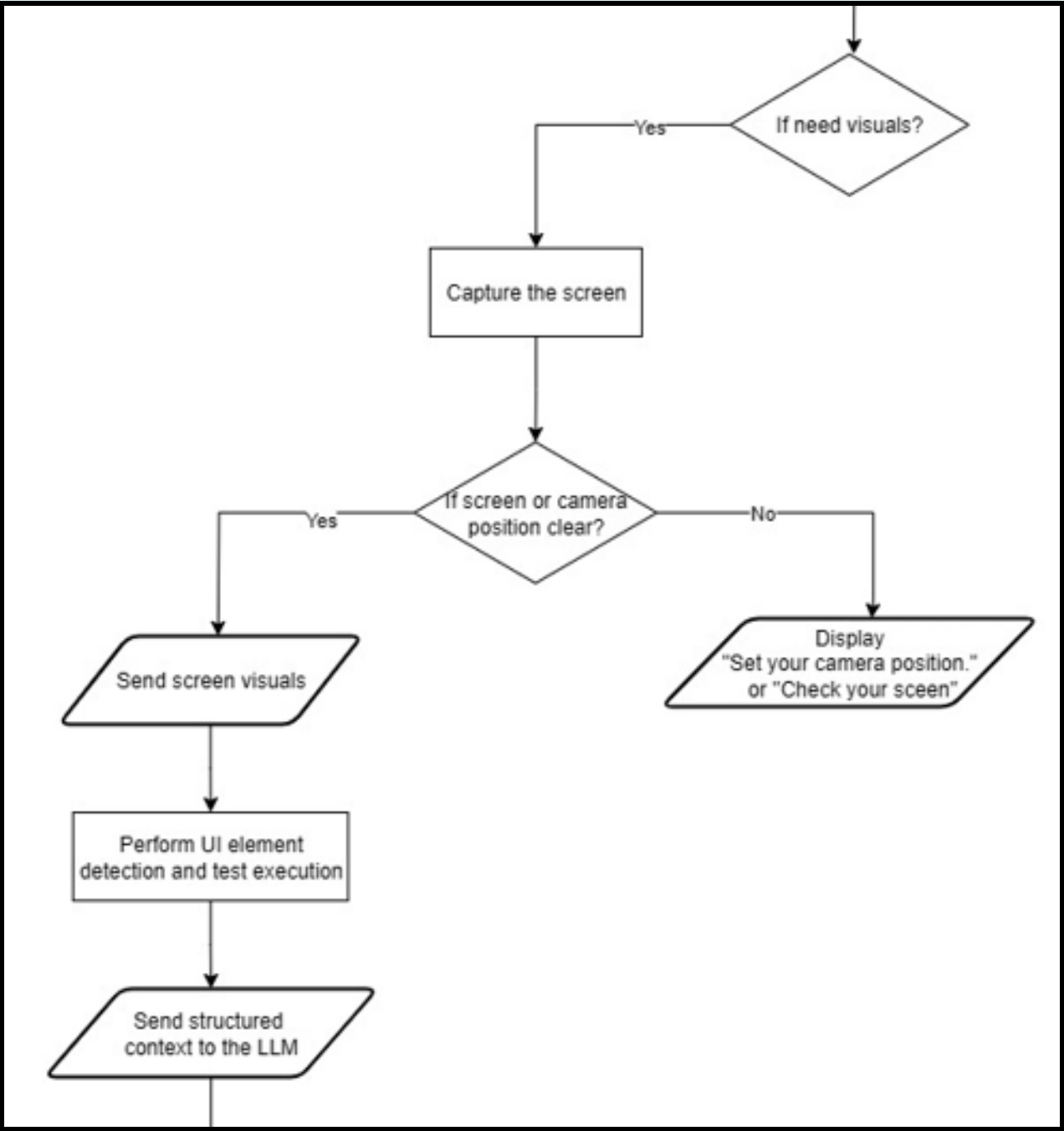


- Data Type → Only visual data (screenshots, screen recordings) captured via external camera; no personal data collected from system memory.
- Anonymization → Sensitive text (e.g., usernames, passwords) blurred or excluded in dataset preparation.
- Data Storage → Images processed in-memory; no long-term storage unless explicitly allowed.
- Ethical Approval → Conforms to software testing ethics no unauthorized access to internal APIs or private system data.

Individual Title: Visual Perception

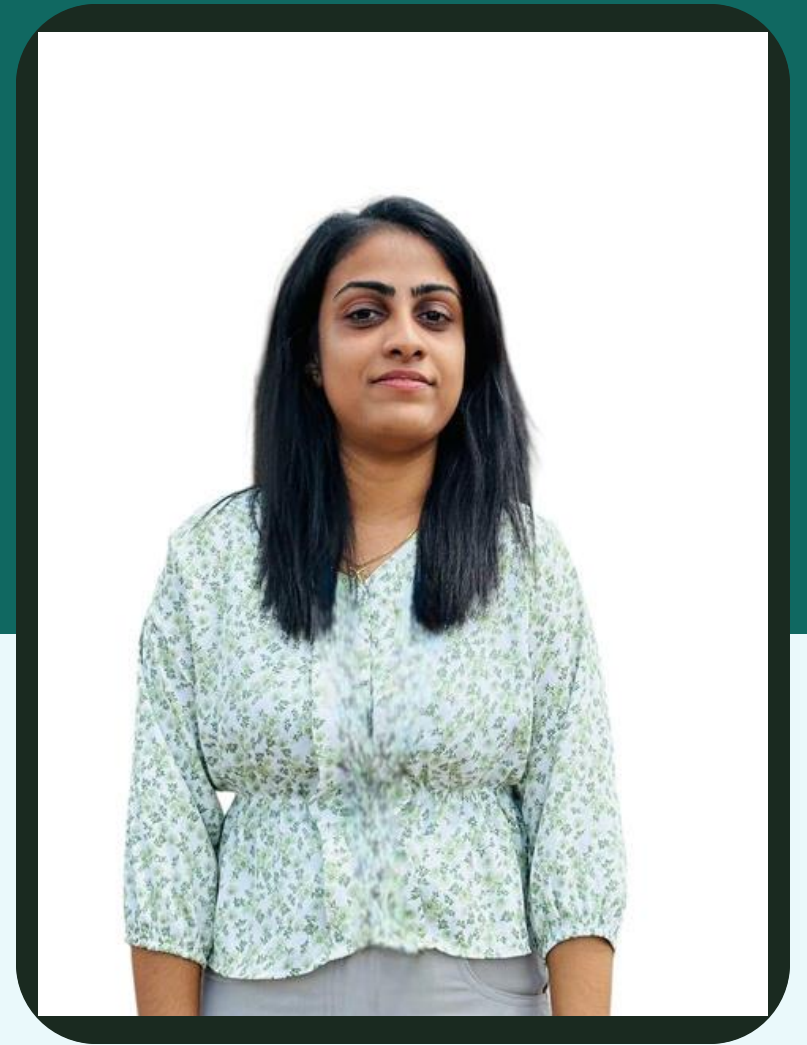
 *Real-World Usage Scenario*

System Overview



Title of the individual component: LLM & Prompt Engineering

Specialization: Software Engineering



Individual Title: LLM & Prompt Engineering

💡 Proven Gap / Creative Solution

- Most tools (like Selenium, Appium) need pre-defined scripts.
- No natural language support – testers must write scripts.
- **Creative solution:** Our solution uses natural language instructions, LLM-generated actions, and adapts dynamically without manual scripting.

🔍 Knowledge Gap / Problem Definition

- Existing automation tools (e.g., Selenium, Appium) require scripted test cases, limiting adaptability.
- No support for natural language instructions.
- Failures occur when UI changes unexpectedly.
- **Problem Definition:** UI automation is rigid, script-based, and breaks with interface changes.

⚠️ Shortcomings of Existing Systems & Proposed Solution

Shortcomings

- Existing tools need scripts.
- Lack natural language support.
- Struggle with changes.
- Require manual prompt setup.

Proposed Solution

- No manual scripting needed.
- Accepts natural language instructions.
- Adapts to dynamic UI changes.
- Prompts created using prompt engineering.

Individual Title: LLM & Prompt Engineering

Key Domains of Knowledge

- 
- NLP – process tester instructions
 - LLMs – generate step-by-step actions
 - Prompt Engineering – create context-aware prompts
 - UI Automation – handle UI elements and events
 - AI Context Awareness – manage dynamic UI changes

Latest Technologies

- 
- Large Language Models (LLMs) – GPT-4, LLaMA
 - Prompt Engineering Tools – LangChain, PromptLayer

Individual Title: LLM & Prompt Engineering

Evaluation & Validation



- Accuracy of Action Plans: LLM steps correctly perform intended UI tasks.
- Natural Language Understanding: Instructions and synonyms correctly interpreted.
- Robustness: Handles dynamic UI changes and unexpected inputs.
- Efficiency: Reduces manual scripting and test time.

Data & Ethical Clearance

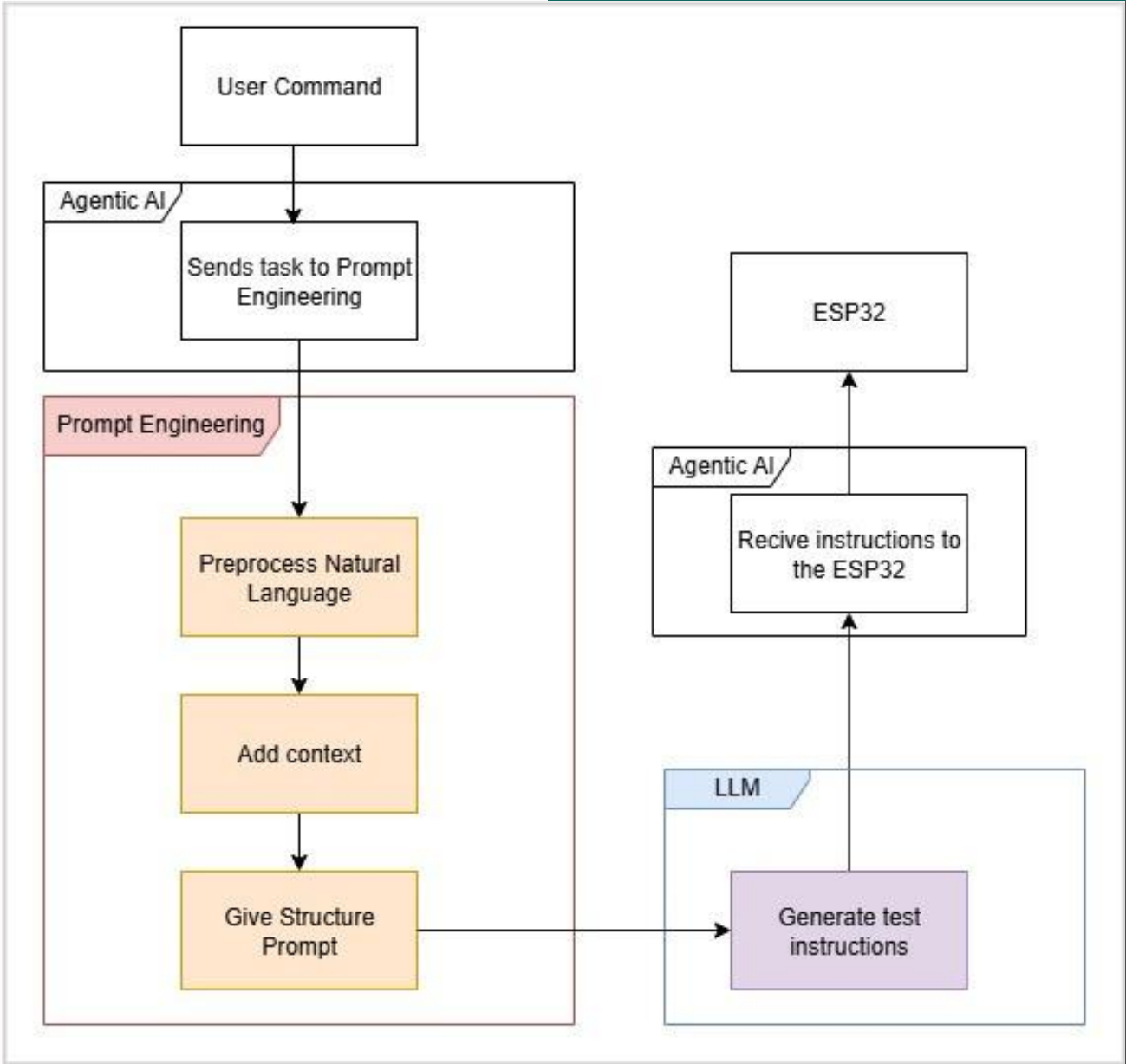
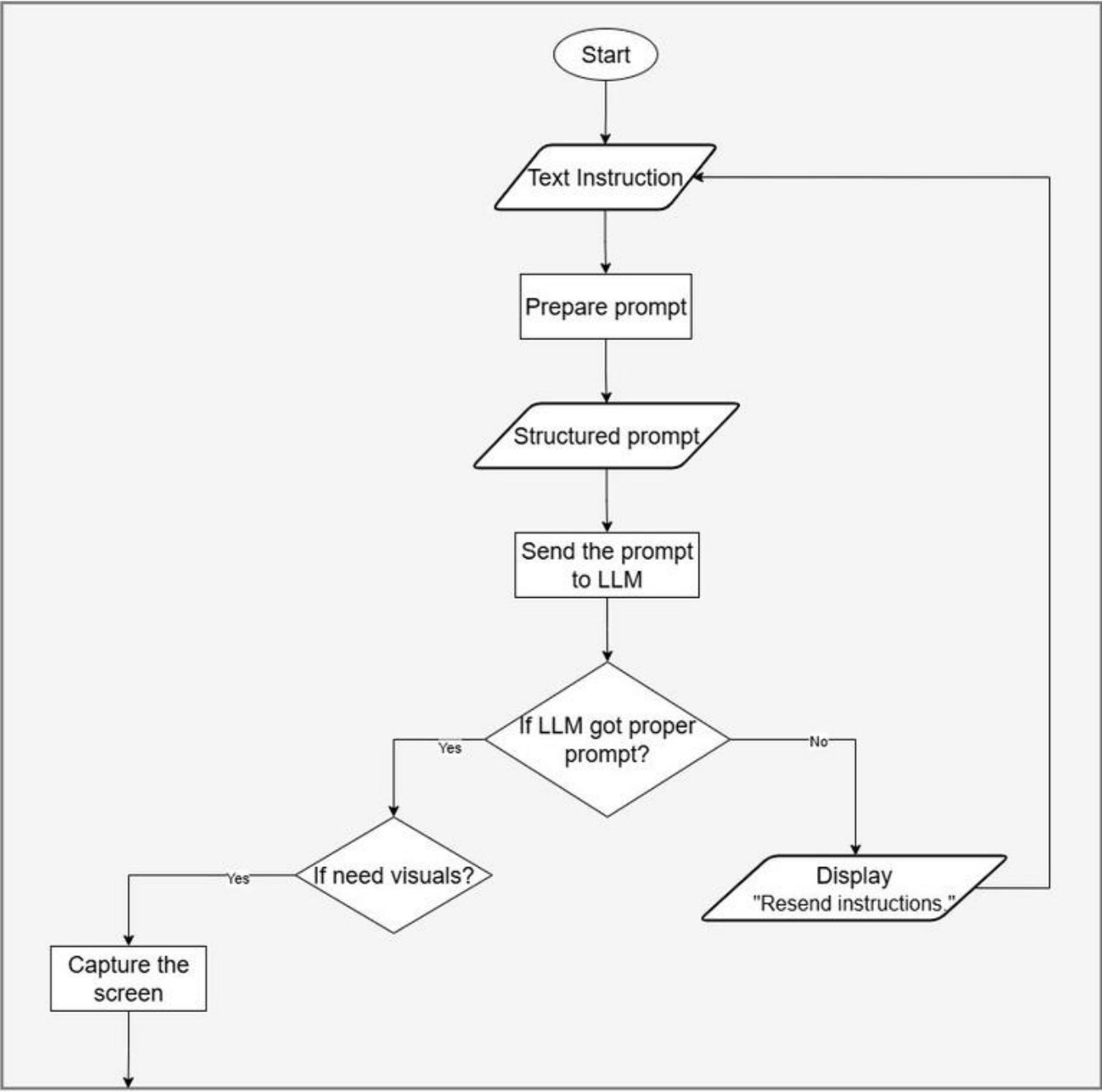


- Data Sources: Use anonymized or synthetic UI data for testing.
- Privacy: Ensure privacy by excluding sensitive user information.
- Ethical AI Use: Use AI responsibly and fairly.
- Compliance: Follow institutional rules and policies.

Individual Title: LLM & Prompt Engineering

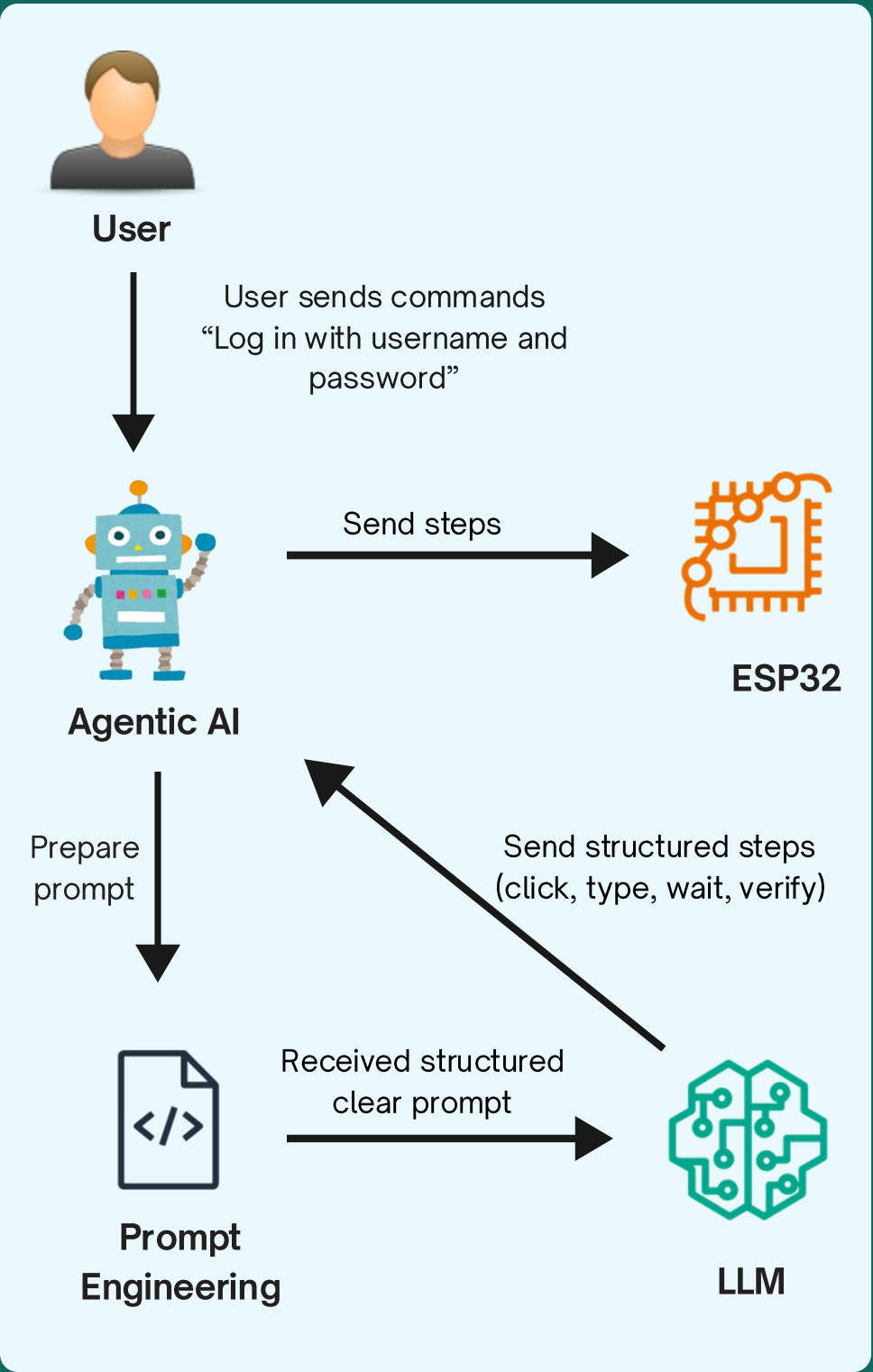
System Overview

The system converts user instructions into executable UI actions, adapts dynamically using visual feedback, and enables platform-independent, black-box automation without internal system access.



Real-World Usage Scenario

- Example: Login to Dashboard with username and password



Title of the individual component: HID Actuation

Specialization: Software Engineering



Individual Title: HID Actuation

💡 Proven Gap / Creative Solution

- Existing HID emulators (e.g., Rubber Ducky, basic Arduino HID) are static, pre-scripted and lack adaptability.
- They cannot provide feedback on execution.

Creative solution: an intelligent HID actuation module that:

- Works in real time, driven by Agentic AI commands.
- Provides bidirectional communication (execution → feedback → retry).

🔍 Knowledge Gap / Problem Definition

- Most existing solutions rely on API access or the ability to inject code into the target application.
- Such approaches fail for legacy, black-box, or restricted software, where internal access is impossible
- How can an ESP32-based HID emulation system reliably reproduce human-like keyboard and mouse interactions?

⚠️ Shortcomings of Existing Systems & Proposed Solution

Shortcomings

- Static execution, no feedback loop
- Limited reliability in dynamic, real-world testing.
- fail in restricted environments.
- Require manual prompt setup.

Proposed Solution

- Dynamic & adaptive
- Execution feedback loop
- Lightweight & undetectable
- No manual scripting needed

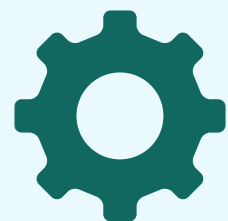
Individual Title: HID Actuation

Key Domains of Knowledge



- Embedded systems & microcontroller programming.
- USB HID protocols.
- Serial communication and real-time feedback handling.
- Feedback & Error Handling Mechanisms
- Real-Time Systems & Timing Control

Latest Technologies



- ESP32-S3 with native USB OTG.
- ESP-IDF and TinyUSB HID library.
- Integration with AI-driven orchestration.(Agentic AI)

Individual Title: HID Actuation

Evaluation & Validation



- Functional testing across Windows, Linux, and macOS.
- Latency: < 300 ms per action
- Accuracy: > 95% execution correctness.
- Reliability: > 85% task success in exploratory testing scenarios

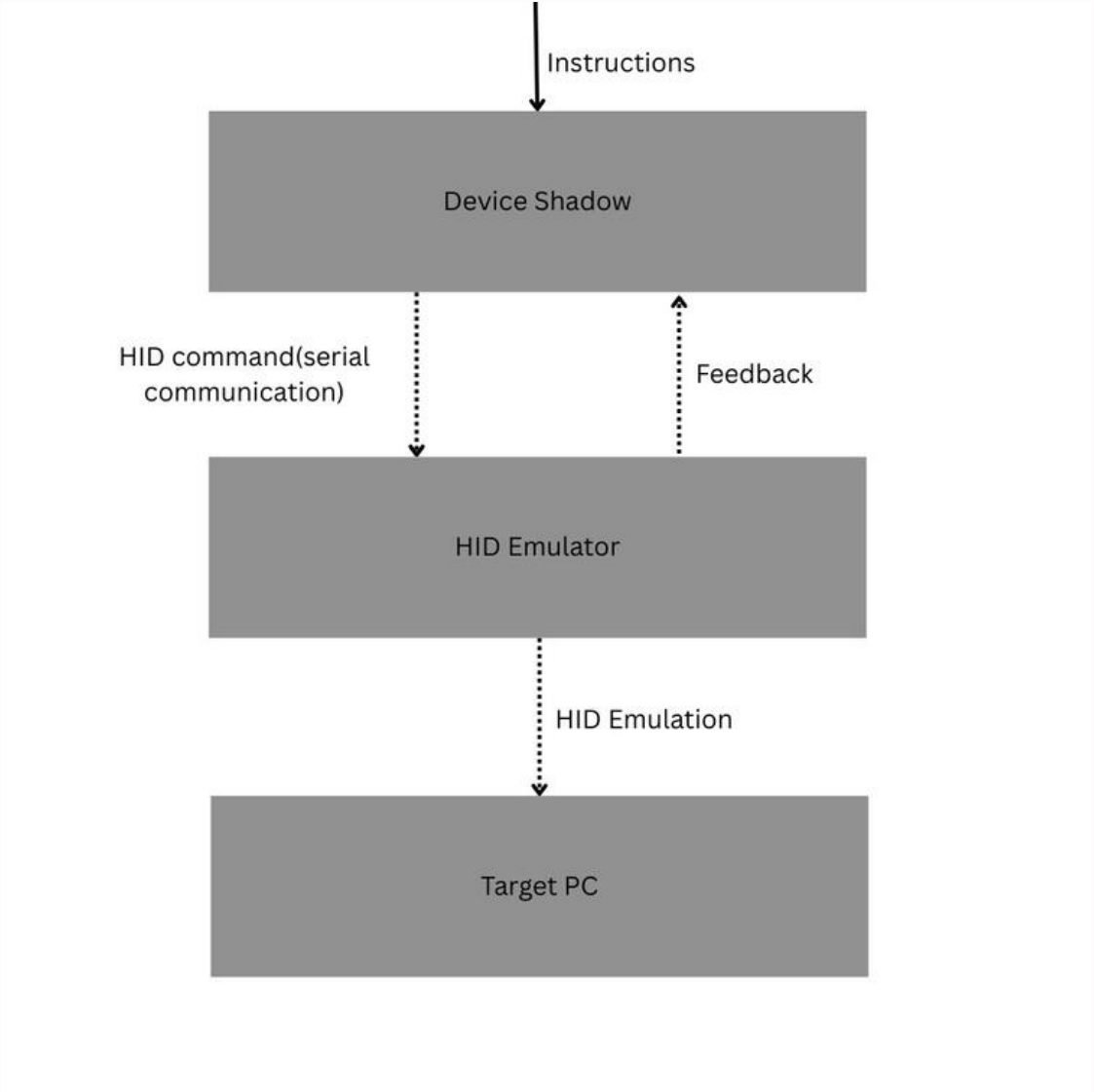
Data & Ethical Clearance



- Data Privacy: The HID module does not store or transmit sensitive user data.
- Ethical Usage: Prevent misuse for malicious input injection by ensuring commands only come from authenticated sources.
- Safety & Compliance.
- Traceability.

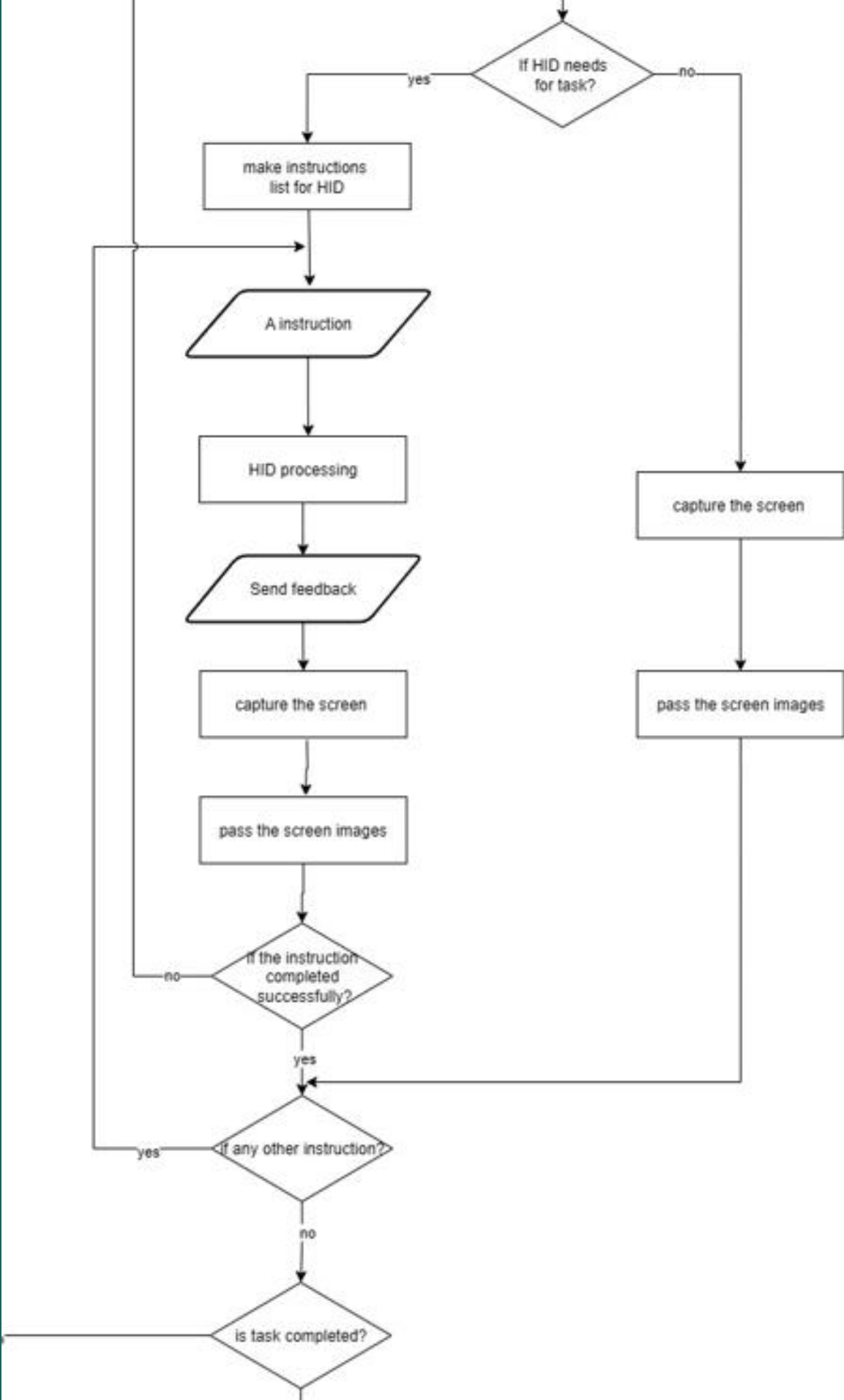
Individual Title: HID Actuation

System Overview



Real-World Usage Scenario

- *Instruction: “Press Enter”.*



Title of the individual component: Agentic AI Logic

Specialization: Software Engineering



Individual Title: Agentic AI Logic

💡 Proven Gap / Creative Solution

- Require internal access or APIs.[4][5]
- Can't adapt dynamically to unexpected UI changes.[7]
- **Creative solution:** Agentic AI acts as an intelligent orchestrator,
 - observing externally,
 - planning dynamically
 - adapting autonomously based on real-time feedback.

🔍 Knowledge Gap / Problem Definition

- Automation systems fail in black-box scenarios where internal data is inaccessible[4].
- Current tools lack adaptive decision-making, causing repeated failures with dynamic UIs.[5]

⚠️ Shortcomings of Existing Systems & ✅ Proposed Solution

Shortcomings

- Dependence on system APIs.[4]
- Lack of adaptability[7]
- Weak reporting[4][5]

Proposed Solution

- External perception with screen capture and HID control.
- Real-time Perceive → Plan → Act → Evaluate loop.
- Integrate logging and Jira for task tracking and reports.

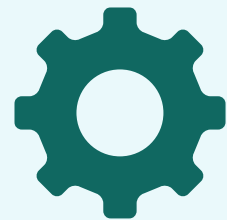
Individual Title: Agentic AI Logic

Key Domains of Knowledge



- AI & Machine Learning → Adaptive Agentic AI
- NLP → Interpret LLM action plans
- Computer Vision → Detect UI changes
- Software Engineering → Orchestration & integration

Latest Technologies



- Agentic AI loop: Perceive → Plan → Act → Evaluate
- LLMs: GPT, LLaMA, Mistral
- Retries & recovery
- Jira API for task reporting & tracking

Individual Title: Agentic AI Logic

Evaluation & Validation



- Validation Approach: Run black-box automation test scenarios on real applications
- Key Metrics:
 - Task Success Rate (completion without human help)
 - Retry Frequency (efficiency of adaptive loop)
 - Decision Accuracy under dynamic UI changes
- Target Performance: $\geq 85\%$ overall success rate across diverse test cases

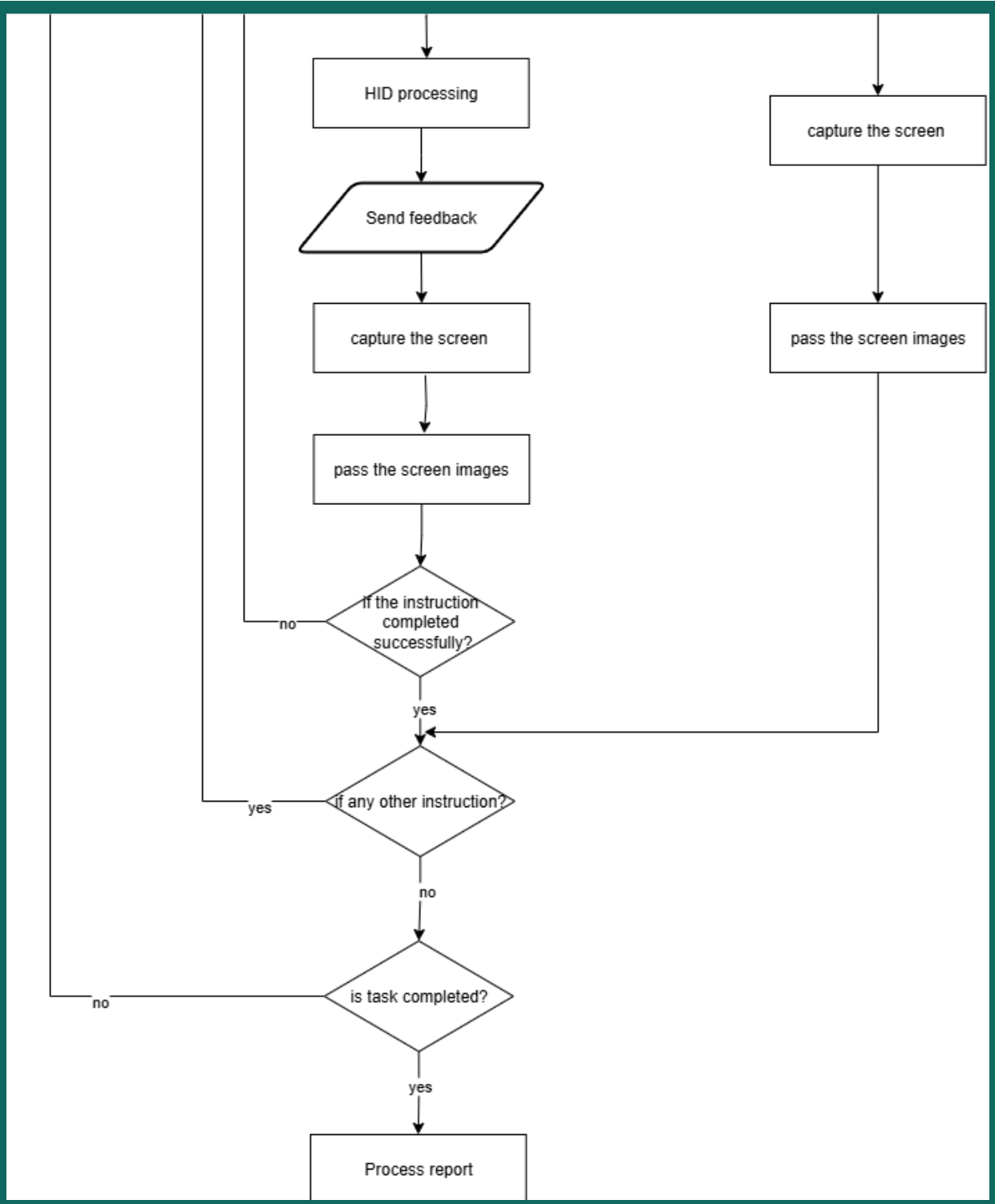
Data & Ethical Clearance



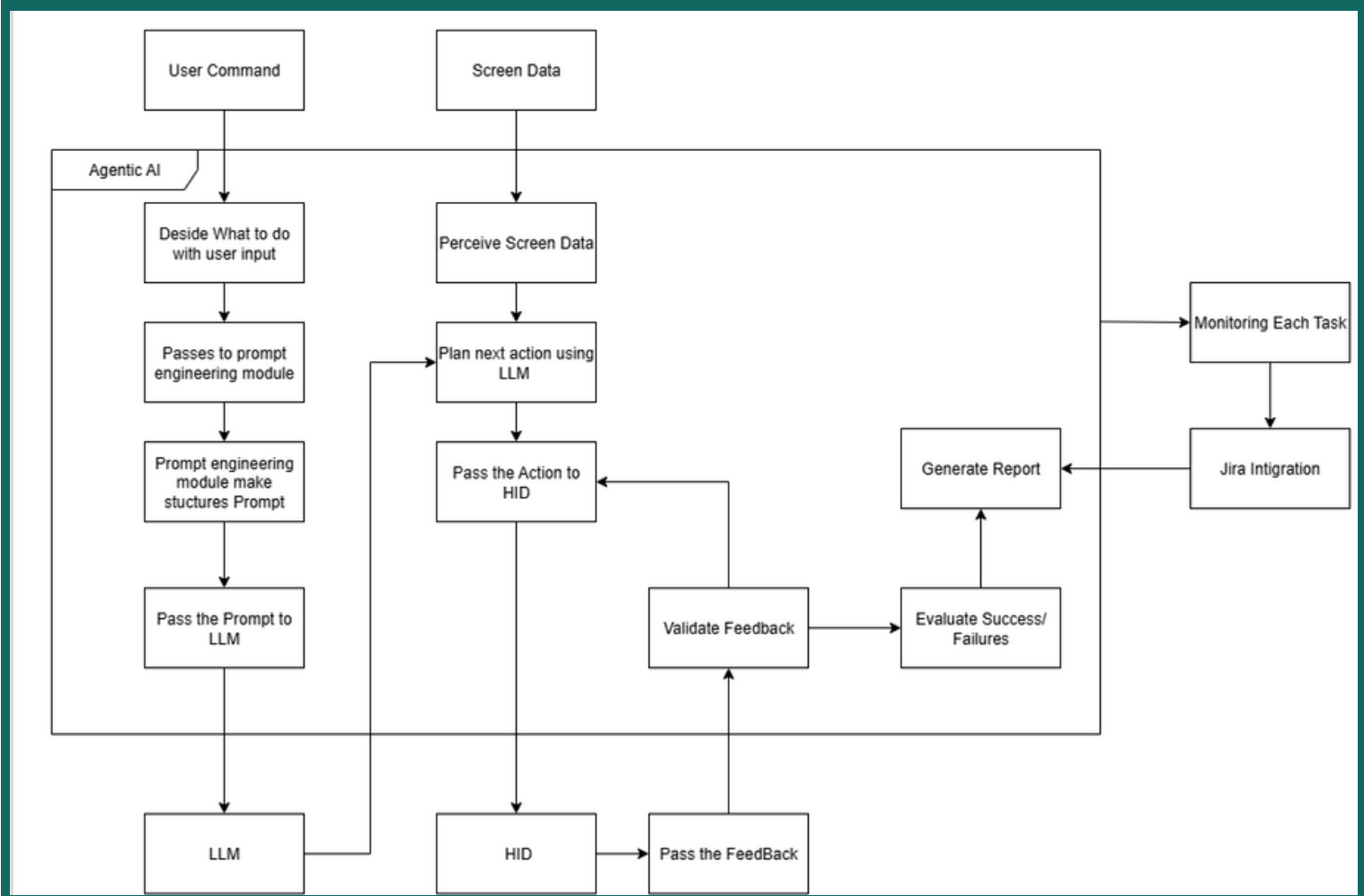
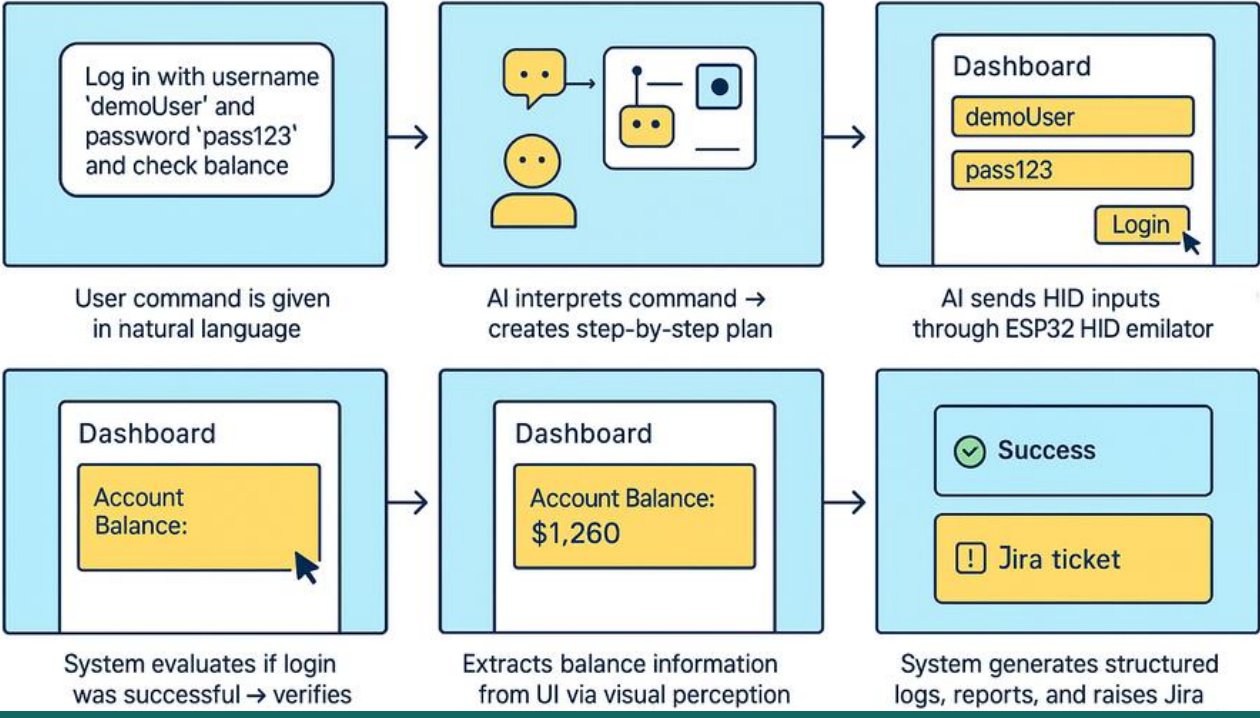
- Data Sources: Screen captures, recordings, synthetic/dummy applications
- Privacy Protection: Not saving sensitive or personal data collected
- Ethical Assurance: External camera-based → non-intrusive, black-box safe
- Intended Use: Software testing & research purposes only (not for malicious activity)

Individual Title: Agentic AI Logic

System Overview



STORYBOARD: AUTOMATED LOGIN & BALANCE CHECK



A Human-Like Agent For Screen-Aware Task Automation

Commercialization & Market Overview

Commercialization:

- Easy-to-use hardware/software for non-technical users
- Integrates with existing workflows
- Pilot tests and demos to attract clients

Total Investment: ~LKR 25,000

- ESP32 & Webcam: LKR 10,000
- Cloud GPU & Software: LKR 10,000
- Miscellaneous & Documentation: LKR 5,000

Cost Recovery:

- Sell 4 prototype kits (~LKR 8,000 each)
- Subscription: LKR 2,000–3,000/month
- Consultancy/pilot projects

Market & Value:

- SMEs, IT firms, educational labs
- Non-technical users benefit from human-like, scalable automation
- Protect IP through software copyright & automation patents

References

- [1] [15] Marvin, G., Hellen, N., Jjingo, D., & Nakatumba-Nabende, J. (2024). Prompt Engineering in Large Language Models (pp. 387–402). https://doi.org/10.1007/978-99-7962-2_30
- [2] Rohit Khankhoje. (2023). An In-Depth Review of Test Automation Frameworks: Types and Trade-offs. International Journal of Advanced Research in Science, Communication and Technology, 55–64. <https://doi.org/10.48175/IJAR SCT-13108>
- [3] S, J. P. B., D, B. K., A, P. S., & D, M. G. (2025). Prompt Engineering for Large Language Models: A Systematic Review and Future Directions. <https://doi.org/10.21203/rs.3.rs-6551106/v1>
- [4] SeleniumHQ, “Selenium Official Documentation,” [Online]. Available: <https://www.selenium.dev/documentation/>. [Accessed: 31-Aug-2025].
- [5] Appium, “Appium Official Documentation,” [Online]. Available: <https://appium.io/docs/en/latest/>. [Accessed: 31-Aug-2025].
- [6] Google Developers, “UIAutomator Documentation,” [Online]. Available: <https://developer.android.com/training/testing/ui-automator>. [Accessed: 31-Aug-2025].
- [7] [UiPath](#)

Survey Link - <https://forms.gle/DZUU8bwG5NWPSGrV9>





*Thank
You*

